

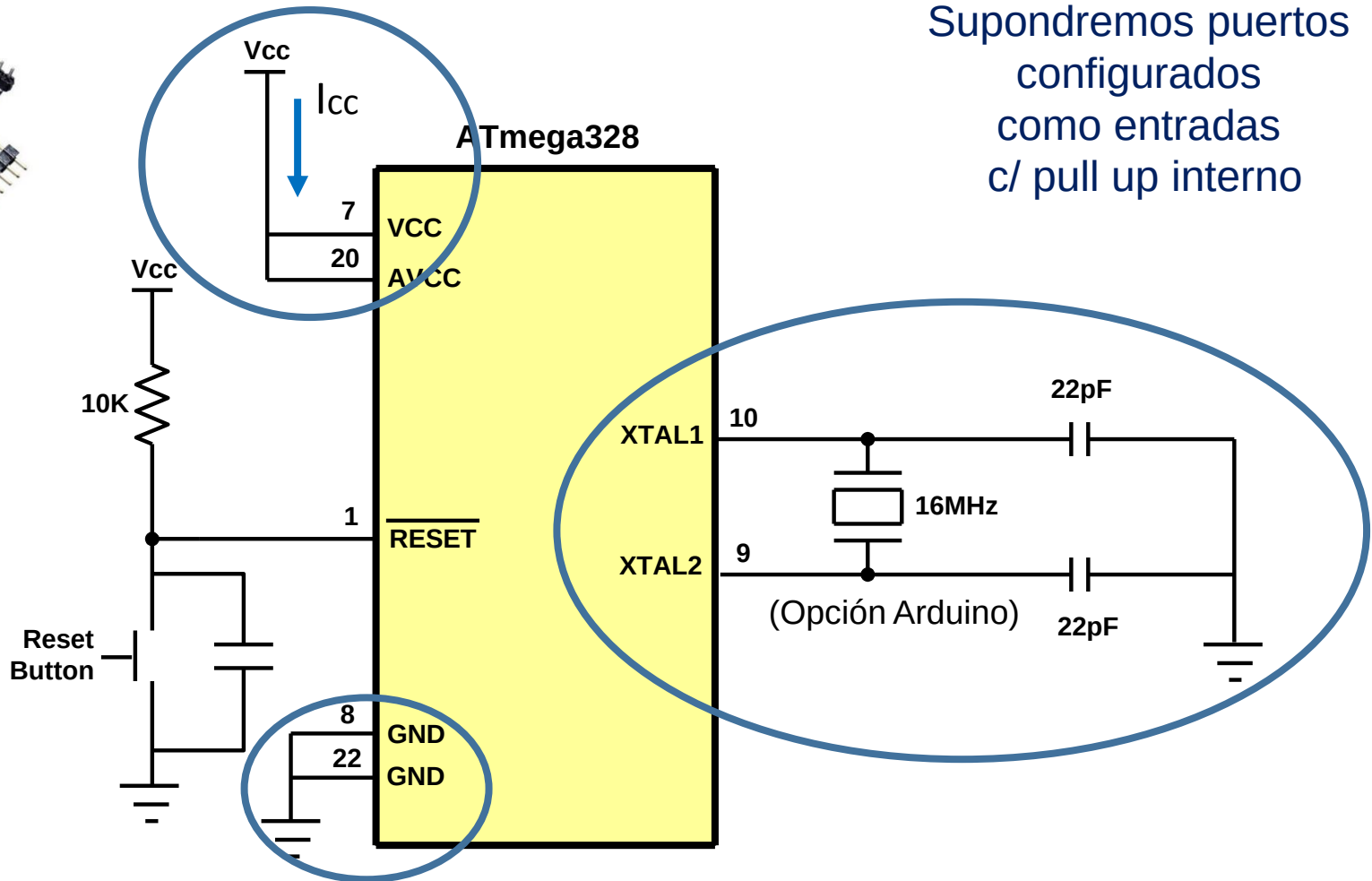
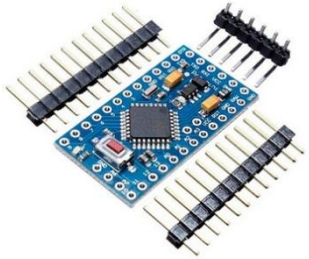
CIRCUITOS DIGITALES Y MICROCONTROLADORES 2026

Facultad de Ingeniería
UNLP

Señales de reloj y
Modos de bajo consumo

Ing. José Juárez

Circuito simple para MCU - AVR



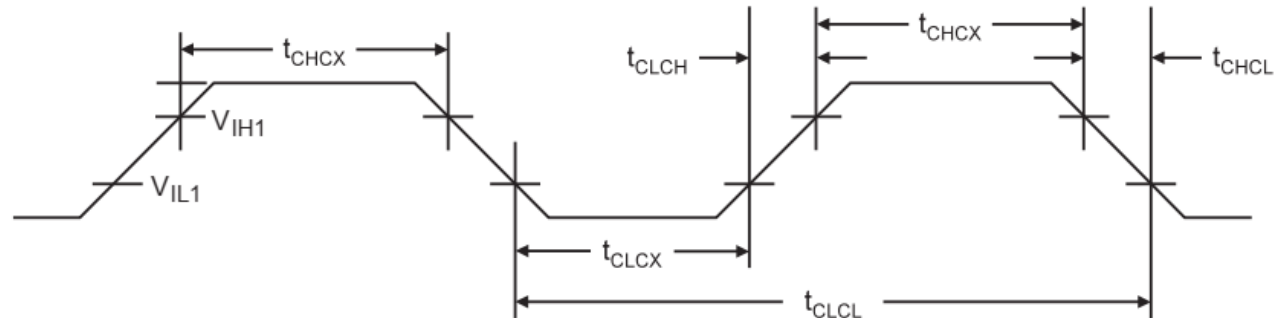
Algunas preguntas para el diseñador...

- ¿qué frecuencia de reloj debo seleccionar?
- ¿qué fuente de reloj puedo seleccionar?
- ¿cómo selecciono una fuente de reloj?
- ¿Por qué hay tantas opciones de reloj?
- ¿Se puede cambiar la frecuencia de reloj durante la ejecución del programa?
- ¿Se puede detener el reloj y luego volver a activarlo?

Señal de reloj

- Especificación:

(para entrada externa)
Hoja de datos Atmega328



- Relación con la Performance del MCU:

El tiempo que tarda en ejecutarse un determinado programa:

$$t_{EXEC} [seg] = total\ de\ instrucciones\ ejecutadas \times CPI \times T_{CLK}$$

- total de instrucciones ejecutadas (depende del programa y del compilador)
- CPI: ciclos por instrucción promedio (depende de la arquitectura)
- $T_{CLK} = 1/f_{CLK}$

[Benchmark CoreMark](#)

- Relación con la Potencia

- **Consumo:** velocidad a la se extrae energía de una fuente
 - ¿Cuánto dura mi batería? O ¿cuanto me vendrá la factura.....?
- **Disipación:** velocidad con la cual la energía se convierte en calor
 - ¿Cuánto se calienta mi circuito?

Disipación

- En los circuitos digitales CMOS:

$$Potencia [Watt] = P_{ESTATICA} + P_{DINAMICA}$$

$$P_{ESTATICA} = V_{CC} I_{FUGA}$$

$$I_{FUGA} = I_{LEAKAGE}$$

$$P_{DINAMICA} = C_L V_{CC}^2 f_{CLK}$$

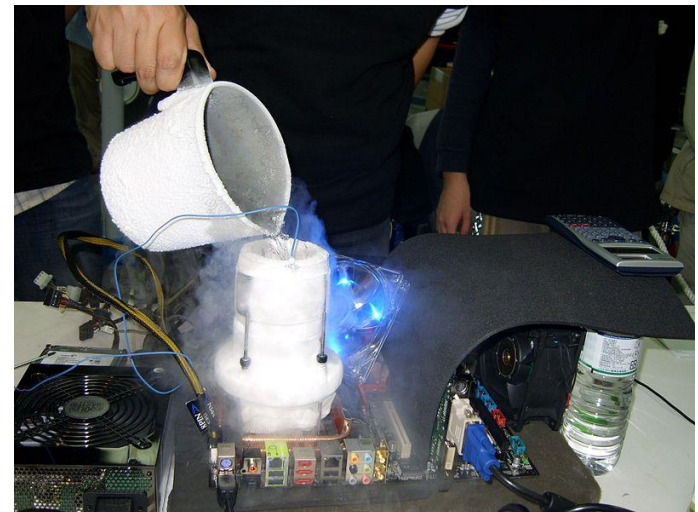
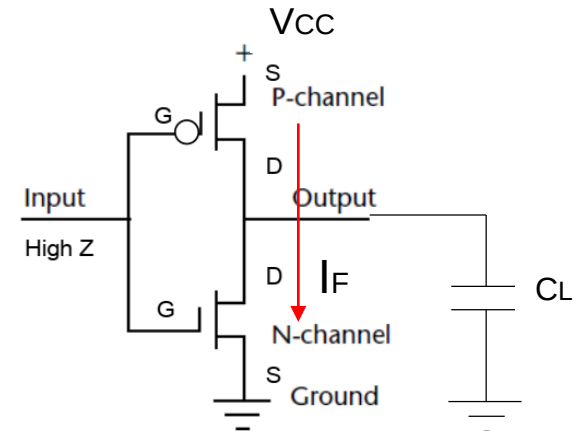
Normalmente $P_E \ll P_D$

- Ejemplo:
 - Circuito integrado con $f_{CLK} = 100\text{MHz}$,
 - $V_{CC} = 5\text{V}$, $C_L = 30\text{ fF/gate}$, 200.000 gates
 - $P_D = 15\text{W}$! $\Rightarrow$ Disipador



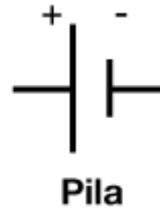
¿ con $f_{CLK} = 10\text{GHz}$?

2004-The **Pentium 4 570 processor** is a 3.8GHz **chip** that will feature the **fastest** clock speed of any **Pentium 4 processor** for an indefinite period of time ($P=115\text{W}$).



https://en.wikipedia.org/wiki/Computer_cooling

Consumo



- Pila tipo botón de 3V
capacidad 500mAh



- Batería recargable Li-Ion de 3.7V
tiene una capacidad de 1500mAh las comunes

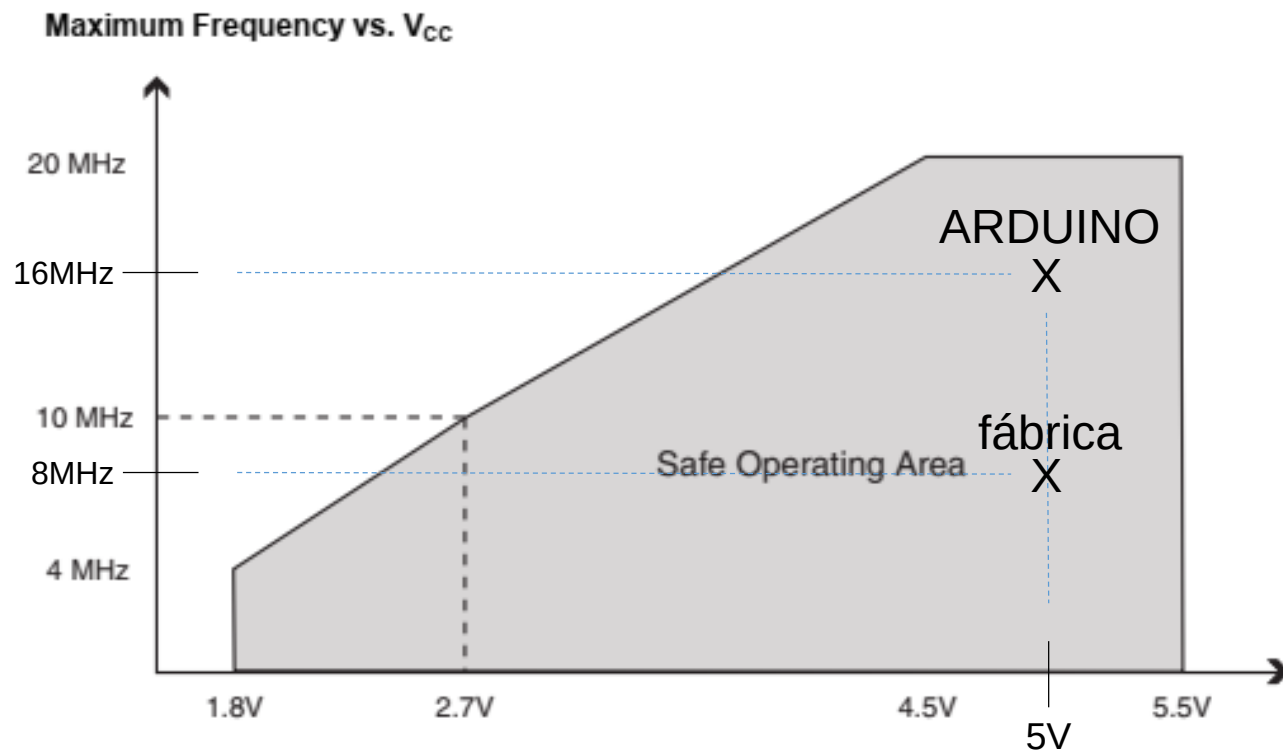


¿Cuál es la corriente I_{cc} que puedo
extraer para que me dure 1 año?

Idealmente 57mA y 171 mA respectivamente

¿qué frecuencia de reloj debo seleccionar?

- Caso de estudio: Atmega328p



¿qué frecuencia de reloj debo seleccionar?

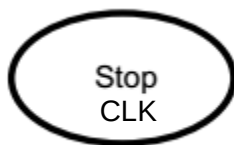
- Caso de estudio: Atmega328p

Table 29-8. ATmega328P DC characteristics - $T_A = -40^{\circ}\text{C}$ to 85°C , $V_{CC} = 1.8\text{V}$ to 5.5V (unless otherwise noted)

Symbol	Parameter	Condition	Min.	Typ. ⁽²⁾	Max.	Units
I_{CC}	Power Supply Current ⁽¹⁾ Icc dinámica	Active 1MHz, $V_{CC} = 2\text{V}$		0.3	0.5	mA
		Active 4MHz, $V_{CC} = 3\text{V}$		1.7	2.5	
		Active 8MHz, $V_{CC} = 5\text{V}$		5.2	9	
		Idle 1MHz, $V_{CC} = 2\text{V}$		0.04	0.15	
		Idle 4MHz, $V_{CC} = 3\text{V}$		0.3	0.7	
		Idle 8MHz, $V_{CC} = 5\text{V}$		1.2	2.7	
	Power-down mode ⁽³⁾ Icc estática	WDT enabled, $V_{CC} = 3\text{V}$		4.2	8	μA
		WDT disabled, $V_{CC} = 3\text{V}$		0.1	2	

@8MHz, @5V, @25°C

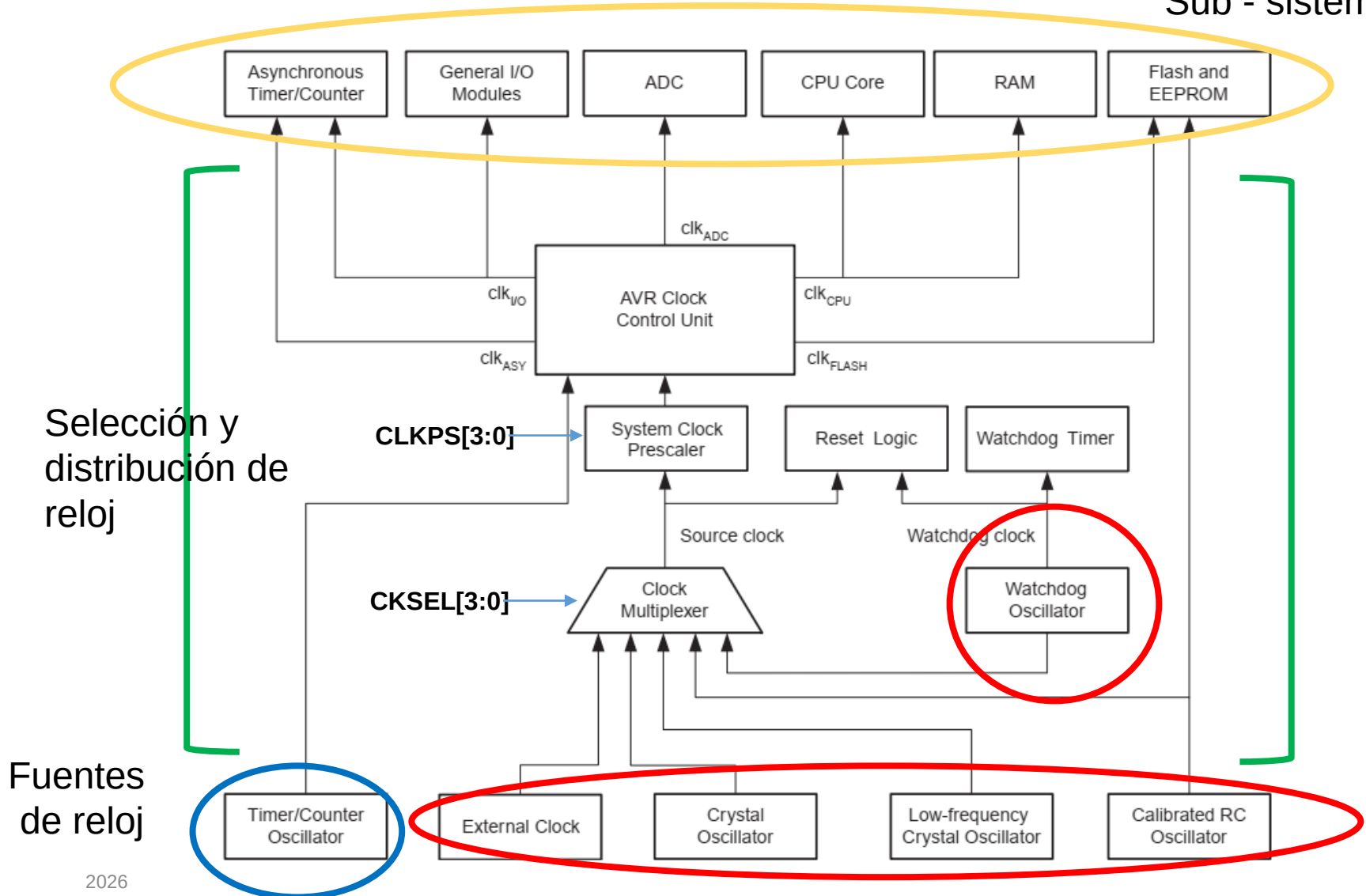
$I_{CC}=9\text{mA}$



$2\mu\text{A}$

¿qué fuente de reloj puedo seleccionar?

Sub - sistemas



Tipos de relojes

- **CPU y FLASH clock (CLK_{CPU} y CLK_{FLASH})**
 - Este es el reloj necesario para el funcionamiento del core AVR. Detener este reloj implica detener todas las operaciones y cálculos de la CPU.
- **I/O clock (CLK_{I/O})**
 - Este es el reloj para la mayoría de los periféricos: puertos, contadores, periféricos de comunicaciones y subsistema de interrupciones externas.
- **ADC clock (CLK_{ADC})**
 - Este reloj permite la operación del periférico ADC independiente a los demás subsistemas. Se recomienda detener CPU e I/O clock para mejorar la exactitud de las conversiones
- **Asynchronous Timer clock (CLK_{ASY})**
 - Este reloj optimizado para trabajar a 32.768kHz, alimenta solamente a un contador para permitir su funcionamiento como RTC (Real Time Counter) incluso con MCU en modo sleep.

¿cómo selecciono una fuente de reloj?

- mediante los fusibles de configuración CKSEL

Los fusibles se programan con el programador ISP

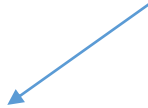


Table 9-1. Device Clocking Options Select⁽¹⁾

Device Clocking Option	CKSEL3...0
Low Power Crystal Oscillator	1111 - 1000
Full Swing Crystal Oscillator	0111 - 0110
Low Frequency Crystal Oscillator	0101 - 0100
Internal 128kHz RC Oscillator (Watchdog clk)	0011
Calibrated Internal RC Oscillator	0010
External Clock	0000
Reserved	0001

Note: 1. For all fuses “1” means unprogrammed while “0” means programmed.

El chip sale desde fábrica configurado con internal RC oscillator a 8.0MHz y con el fuse CKDIV8 programado, resultando 1.0MHz system clock.

¿Para qué necesito tantas opciones de reloj?

• Crystal Oscillator

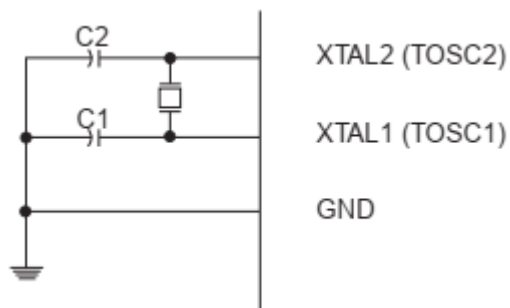
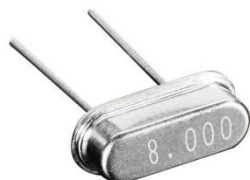


Table 9-3. Low Power Crystal Oscillator Operating Modes⁽³⁾

Frequency Range (MHz)	Recommended Range for Capacitors C1 and C2 (pF)	CKSEL3...1 ⁽¹⁾
0.4 - 0.9	–	100 ⁽²⁾
0.9 - 3.0	12 - 22	101
3.0 - 8.0	12 - 22	110
8.0 - 16.0	12 - 22	111

Table 9-5. Full Swing Crystal Oscillator operating modes

Frequency Range ⁽¹⁾ (MHz)	Recommended Range for Capacitors C1 and C2 (pF)	CKSEL3...1
0.4 - 20	12 - 22	011

Table 9-8. Capacitance for Low-frequency Oscillator

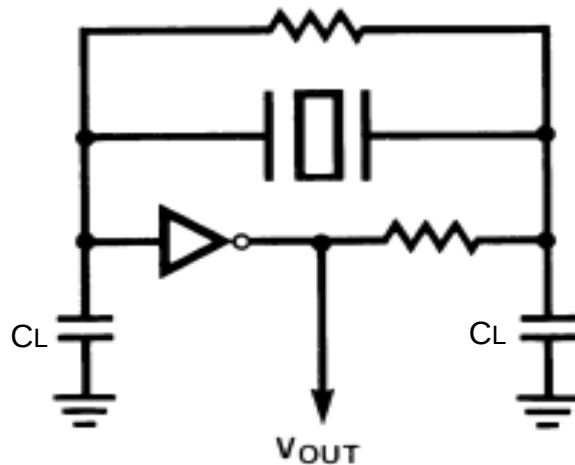
Device	32kHz Osc. Type
ATmega48A/PA/88A/PA/168A/PA/328/P	System Osc.
	Timer Osc.

Low frequency:
32.768 KHz

Si se selecciona Timer Osc (Tosc1 y Tosc2) el reloj de la CPU debe ser el interno

¿Para qué necesito tantas opciones de reloj?

• Crystal Oscillator



AN2648-Selecting_Testing-32KHz-Crystal-Osc-for-AVR-MCUs

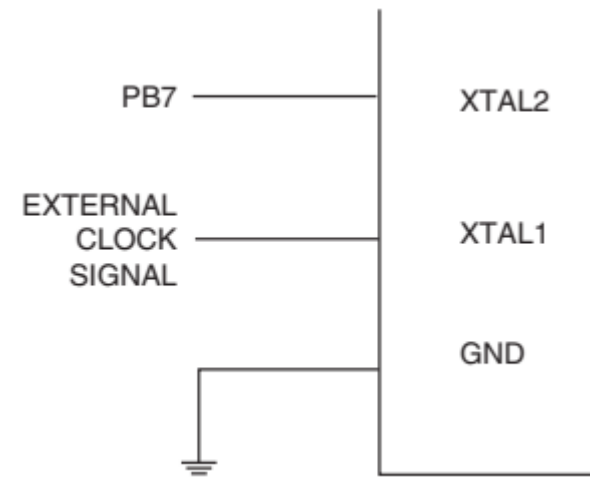
Un cristal típico tiene muy buena exactitud y una estabilidad en frecuencia de $\pm 100\text{ppm}$

Reemplazamos el cristal de cuarzo por su modelo eléctrico

$$f_{\text{CLK}} = 8.000.000 \pm 800\text{Hz}$$



Osciladores a Cristal
Diferentes frecuencias



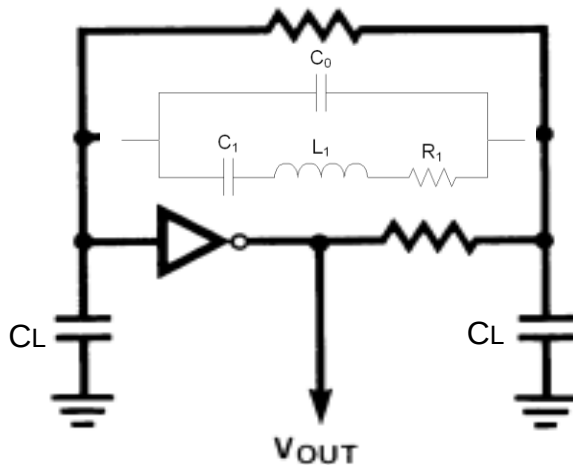
• External Clock

Table 9-15. Crystal Oscillator Clock Frequency

Frequency	CKSEL3...0
0 - 20MHz	0000

¿Para qué necesito tantas opciones de reloj?

• Crystal Oscillator



AN2648-Selecting_Testing-32KHz-Crystal-Osc-for-AVR-MCUs

Un cristal típico tiene muy buena exactitud y una estabilidad en frecuencia de $\pm 100\text{ppm}$

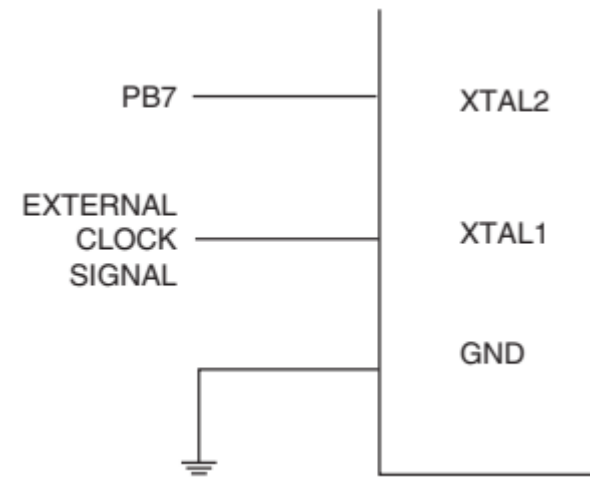
$$f_s = \frac{1}{2\pi\sqrt{L_1 C_1}} \sqrt{1 + \frac{C_1}{C_0}}$$

$$\Delta f = f_s \left(\frac{C_1}{2(C_0 + C_L)} \right)$$

$$f_{\text{CLK}} = 8.000.000 \pm 800\text{Hz}$$



Osciladores a Cristal
Diferentes frecuencias



• External Clock

Table 9-15. Crystal Oscillator Clock Frequency

Frequency	CKSEL3...0
0 - 20MHz	0000

¿Para qué necesito tantas opciones de reloj?

- **Calibrated Internal RC Oscillator**

Table 9-11. Internal Calibrated RC Oscillator Operating Modes

Frequency Range ⁽²⁾ (MHz)	CKSEL3...0
7.3 - 8.1	0010 ⁽¹⁾

- **128kHz Internal Oscillator**

Table 9-13. 128kHz Internal Oscillator Operating Modes

Nominal Frequency ⁽¹⁾	CKSEL3...0
128kHz	0011

Note: 1. Note that the 128kHz oscillator is a very low power clock source, and is not designed for high accuracy.

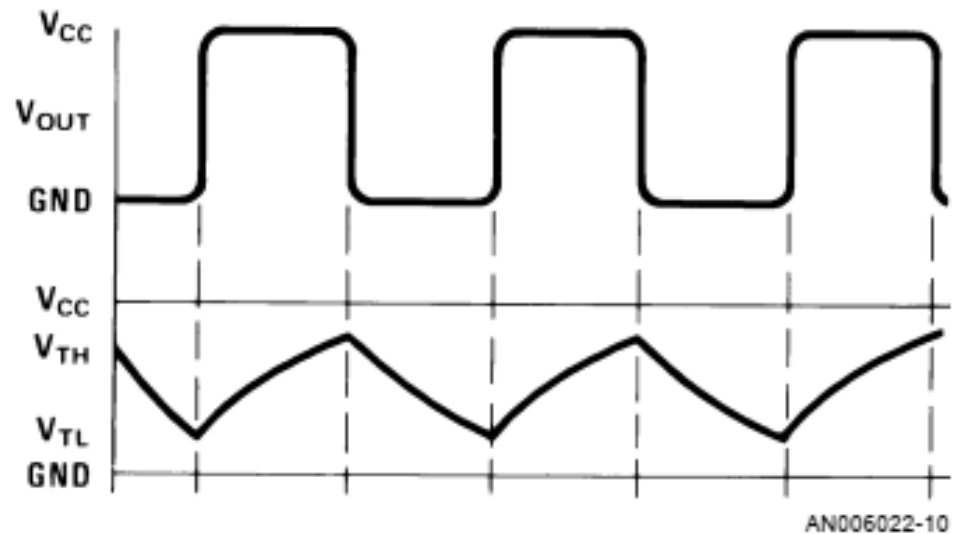
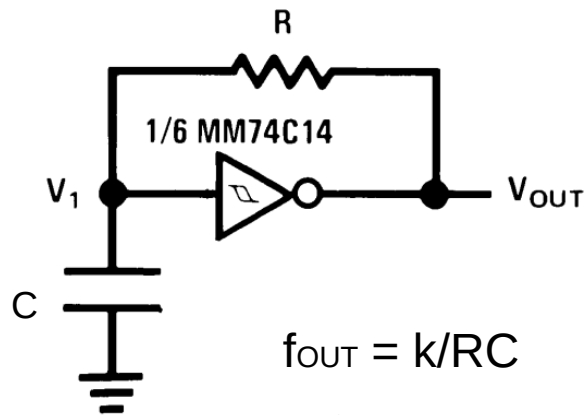
- **Tolerancia:**

Table 29-9. Calibration Accuracy of Internal RC Oscillator

	Frequency	V _{CC}	Temperature	Calibration Accuracy
Factory Calibration	8.0MHz	3V	25°C	±10%
User Calibration	7.3 - 8.1MHz	1.8V - 5.5V	-40°C - 85°C	±1%

¿Para qué necesito tantas opciones de reloj?

- RC Oscillator



Si tomamos la especificación de la hoja de datos
exactitud en la frecuencia de $\pm 10\%$

$$f_{CLK} = 8.000.000 \pm 800000 \text{ Hz}$$

¿Se puede cambiar la frecuencia de reloj durante la ejecución del programa?

- **System Clock Prescaler**

El ATmega328P tiene un prescalador del reloj principal, que permite dividirlo según se configure el registro **CLKPR**.

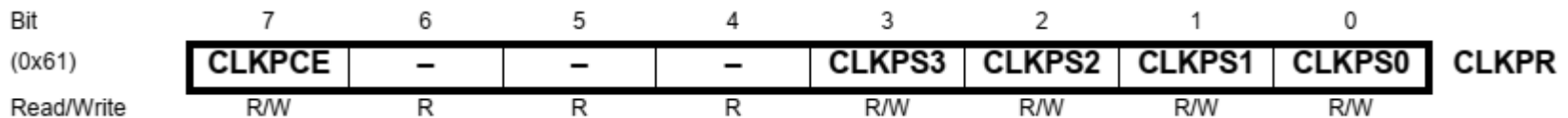


Table 9-17. Clock Prescaler Select

CLKPS3	CLKPS2	CLKPS1	CLKPS0	Clock Division Factor
0	0	0	0	1
0	0	0	1	2
0	0	1	0	4
0	0	1	1	8
0	1	0	0	16
0	1	0	1	32
0	1	1	0	64
0	1	1	1	128
1	0	0	0	256

Clock change enable

Un fuse por defecto lo programa en 8

¿Se puede detener el reloj y luego activarlo?

- Los modos de bajo consumo permiten el ahorro de energía al “apagar” módulos de un MCU, cuyo uso no es requerido en un sistema. En la arquitectura AVR se simplifica el manejo de los modos de bajo consumo por la forma en que distribuye su señal de reloj, de manera que si un módulo no se va a requerir para alguna aplicación, simplemente se anula su señal de reloj, con lo cual el módulo no trabaja y por lo tanto, no consume energía.

Table 10-1 Active Clock Domains and Wake-up Sources in the Different Sleep Modes

Active Clock Domains						Oscillators		Wake-up Sources					
	clk _{CPU}	clk _{FLASH}	clk _{IO}	clk _{ADC}	clk _{ASY}	Main Clock Source Enabled	Timer Oscillator Enabled	INT1, INT0 and Pin Change	TWI Address Match	Timer2	SPM/EEPROM Ready	ADC	WDT
Idle			X	X	X	X	X ⁽²⁾	X	X	X	X	X	X
ADC Noise Reduction				X	X	X	X ⁽²⁾	X	X	X ⁽²⁾	X	X	X
Power-down								X	X				X
Power-save					X		X ⁽²⁾	X	X	X			X
Standby ⁽¹⁾						X		X	X				X

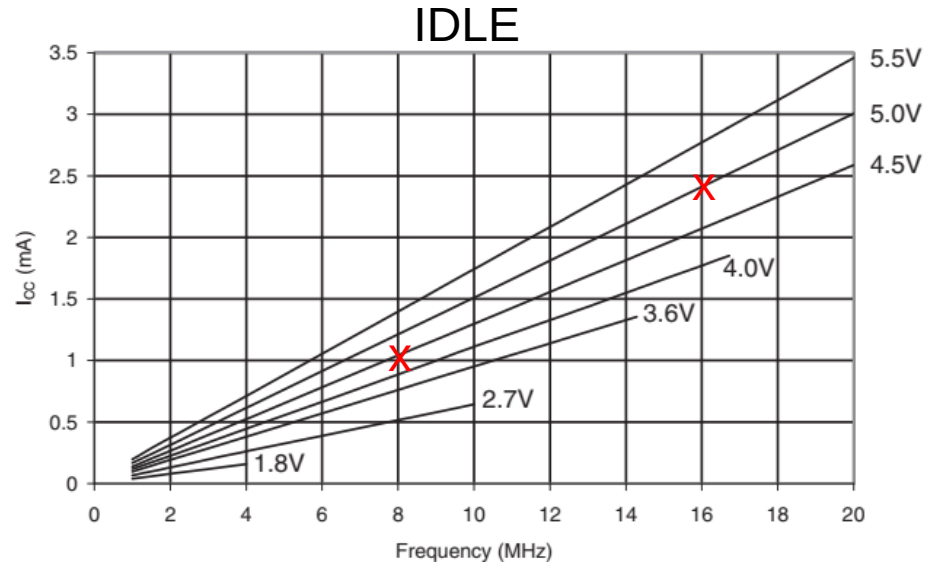
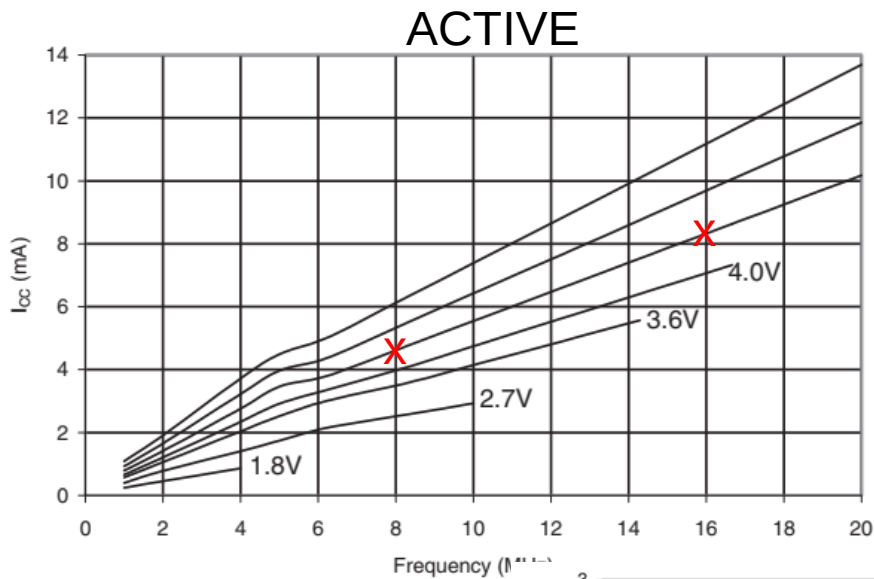
Modos de bajo consumo

- **Modo Idle (ocioso):** En este modo todos los recursos del MCU trabajan, pero la CPU no ejecuta instrucciones porque no tienen señal de reloj. El suministro del reloj principal y del oscilador del temporizador están activos. El hecho de mantener los recursos activos hace que, prácticamente cualquier evento de los diferentes recursos provoque una salida del modo de reposo.
- **Modo Power Down:** En este modo no hay reloj en los módulos de recursos y tampoco están activas las fuentes de oscilación, por lo tanto, es el modo con el menor consumo de energía. El MCU puede ser reactivado por eventos en la interfaz I2C o por las interrupciones externas. Una posible aplicación para este modo es un control remoto, porque casi siempre está inactivo, sólo cuando se presiona una tecla del control remoto el MCU se despierta, emite el código seleccionado y regresa al modo de reposo.
- **Modo Power Save:** En este modo sólo se tiene al reloj asíncrono, con su correspondiente oscilador. Por lo que prácticamente sólo se mantiene activo el temporizador 2, sincronizado con una fuente de reloj externa. Esto hace al modo ideal para aplicaciones que involucren un reloj de tiempo real. La salida del modo es posible con eventos en la interfaz de dos hilos, interrupciones externas o del temporizador 2.
- **Modo Standby:** Este modo es muy similar al modo de baja potencia, la única diferencia es que en este modo se mantiene el suministro del reloj principal. Con ello, el dispositivo despierta en 6 ciclos de reloj.
- **Modo de reducción de ruido en el ADC:** En este modo únicamente trabaja el ADC y el oscilador asíncrono para el temporizador 2. Ambos suministros de reloj están activos, por ello, este modo es adecuado para aplicaciones que requieren el monitoreo de parámetros analógicos en periodos preestablecidos de tiempo. La salida del modo es posible por eventos en los recursos principales.

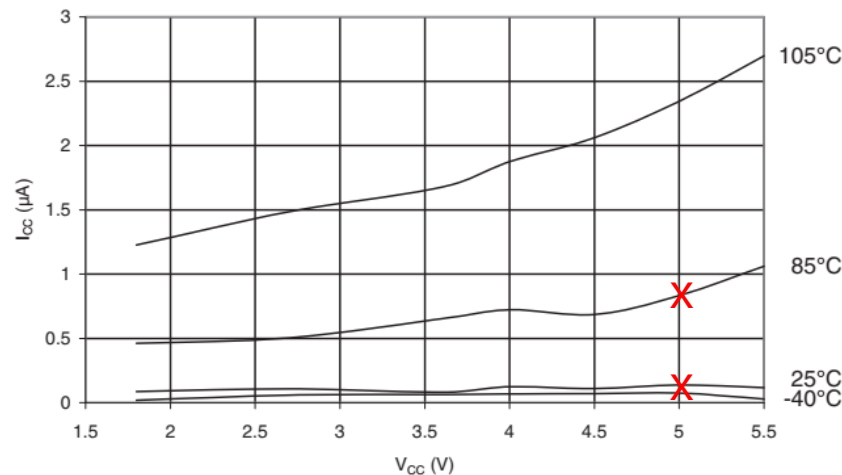
Modos de bajo consumo

- $I_{DD} \propto C_L V_{CC} f_{CLK}$

- Crecimiento lineal del consumo de la CPU con la frecuencia de operación



POWER DOWN



- Crecimiento exponencial de la corriente estática (pérdida) de la CPU con la Temperatura de operación.

Modos de bajo consumo

- Los periféricos del MCU también impactan en el consumo

PRR – Power Reduction Register

Bit	7	6	5	4	3	2	1	0	
(0x64)	PRTWI	PRTIM2	PRTIM0	–	PRTIM1	PRSPI	PRUSART0	PRADC	PRR
Read/Write	R/W	R/W	R/W	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

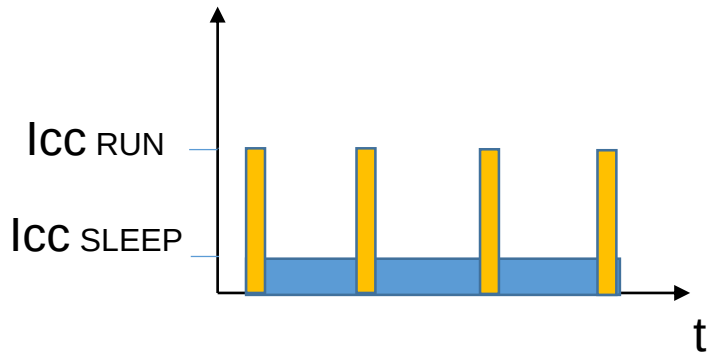
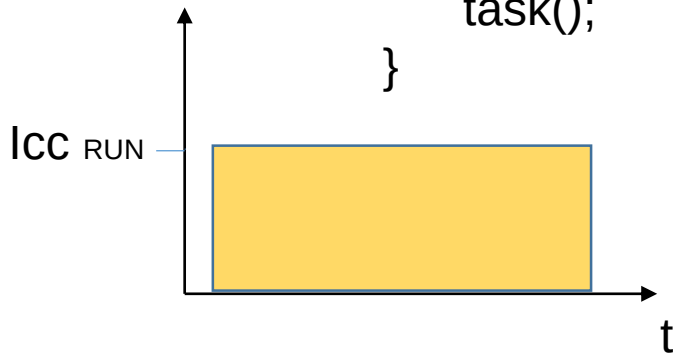
Table 31-15. ATmega328P: Additional Current Consumption for the different I/O modules

PRR bit	Typical numbers		
	$V_{CC} = 2V, F = 1MHz$	$V_{CC} = 3V, F = 4MHz$	$V_{CC} = 5V, F = 8MHz$
PRUSART0	3.20 μA	22.17 μA	100.25 μA
PRTWI	7.34 μA	46.55 μA	199.25 μA
PRTIM2	7.34 μA	50.79 μA	224.25 μA
PRTIM1	6.19 μA	41.25 μA	176.25 μA
PRTIM0	1.89 μA	14.28 μA	61.13 μA
PRSPI	6.94 μA	43.84 μA	186.50 μA
PRADC	8.66 μA	61.80 μA	295.38 μA

$$I_{ccTotal} = I_{ccCPU} + I_{ccPRx} + I_{ccI/O} \leftarrow \text{Leds, drivers, etc}$$

Estrategias de bajo consumo – Arq. del software

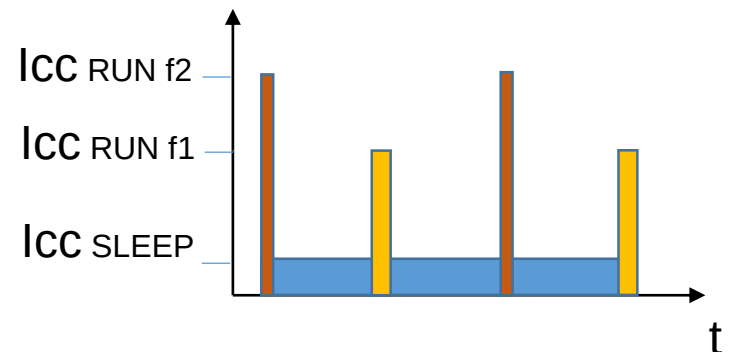
```
while(1) {  
    task();  
}
```



```
while(1) {  
    task();  
    sleep();  
}
```

← Wakeup con interrupción

https://en.wikipedia.org/wiki/Dynamic_frequency_scaling



```
while(1) {  
    change_clk(2);  
    task(x);  
    sleep();  
    change_clk(1);  
    task(y);  
    sleep();  
}
```

Bibliografía complementaria

- *The AVR microcontroller & Embedded Systems*. Mazidi, Naimis. (CH8 y APENDICE C)
- *Los Microcontroladores AVR de ATMEL*. Felipe Espinoza (CH3 y CH8)
- Hoja de datos ATMEGA328P (características eléctricas)